

JAVA

OOP Part 1: Herança & Classes

Classes • Objetos • Construtores • Herança • POJO • Override

Apostila completa com explicações detalhadas,
exemplos de código e mini-projects práticos.

CONTEÚDO DA APOSTILA

- 01 Introdução a Classes e Objetos
- 02 null e Valores Padrão
- 03 Getters e Setters — Encapsulamento
- 04 Construtores e Sobrecarga
- 05 this() vs super() — Constructor Chaining
- 06 Herança com extends
- 07 Override de Métodos
- 08 Static vs Instance e POJO
- 09 java.lang.Object
- 10 Overloading vs Overriding
- 11 Mini-Projects Práticos

1

Classes e Objetos

Blueprint, instâncias, referências e valores padrão

A **Programação Orientada a Objetos (OOP)** é o paradigma central do Java. Entender o que são classes, objetos e referências é o ponto de partida para tudo o mais. Esta seção cobre esses fundamentos com profundidade — incluindo os detalhes que costumam confundir iniciantes.

O que é uma Classe?

Uma **classe** é um *blueprint* (planta, modelo) — ela descreve quais dados (campos/atributos) e quais comportamentos (métodos) os objetos criados a partir dela terão. A classe em si não ocupa memória heap — é apenas uma definição. Pense na analogia de uma planta arquitetônica: a planta não é a casa; é o projeto a partir do qual várias casas idênticas podem ser construídas.

Um **objeto** (ou **instância**) é criado a partir de uma classe usando o operador `new`. Cada objeto tem seu próprio espaço de memória no heap e seus próprios valores de campos. Ao atribuir um objeto a uma variável, você não está copiando o objeto — está copiando a **referência** (o endereço de memória onde o objeto vive). Isso significa que duas variáveis podem apontar para o mesmo objeto.

```
// Criando uma instância da classe House
House blueHouse = new House("blue");

// Duas referências para o MESMO objeto:
House anotherHouse = blueHouse; // copia a referência, não o objeto
// Modificar via anotherHouse afeta blueHouse também!
```

■ Analogia

Analogia: a classe é a planta de uma casa; cada objeto é uma casa construída a partir dessa planta. Você pode construir quantas casas quiser do mesmo projeto.

null e Valores Padrão dos Campos

`null` é uma palavra-chave especial do Java que representa a ausência de um objeto. Uma variável de referência que não foi inicializada aponta para `null` — ela tem um tipo declarado, mas nenhum objeto atribuído. Tentar chamar qualquer método em uma referência `null` lança `NullPointerException` em tempo de execução — um dos erros mais comuns em Java.

Campos de instância recebem valores padrão automaticamente quando um objeto é criado: tipos numéricos inteiros (`byte`, `short`, `int`, `long`, `char`) recebem `0`; tipos de ponto flutuante (`float`, `double`)

recebem **0.0**; boolean recebe **false**; e qualquer tipo de referência (incluindo String) recebe **null**. Variáveis locais (dentro de métodos) *não* recebem valores padrão — devem ser inicializadas antes do uso.

```
public class Aluno {  
    String nome;    // null por padrão  
    int idade;      // 0 por padrão  
    boolean ativo;  // false por padrão  
    double nota;    // 0.0 por padrão  
}
```

■ Cuidado com null

Tentar chamar um método em uma variável null lança NullPointerException em tempo de execução. Sempre verifique se uma referência é null antes de usá-la: `if (obj != null) { obj.metodo(); }`

Getters e Setters — Por que Usá-los?

O princípio de **encapsulamento** diz que os campos de uma classe devem ser private — inacessíveis diretamente de fora. O acesso se dá exclusivamente através de métodos públicos chamados **getters** (para ler) e **setters** (para escrever). Isso traz controle total: o setter pode validar os dados antes de atribuí-los, e a implementação interna pode mudar sem afetar o código cliente.

Por convenção, getters se chamam `get` + nome do campo em CamelCase (ex: `getNome()`). Para boolean, usa-se `is` (ex: `isAtivo()`). Setters se chamam `set` + campo. A palavra-chave `this` dentro do setter resolve a ambiguidade quando o parâmetro tem o mesmo nome do campo.

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public String getNome() { return nome; }  
    public void setNome(String nome) { this.nome = nome; }  
  
    public int getIdade() { return idade; }  
    public void setIdade(int idade) {  
        if (idade >= 0) this.idade = idade; // valida antes de atribuir!  
    }  
}
```

2

Construtores

Constructor, sobrecarga e constructor chaining

O que é um Construtor?

Um **construtor** é um bloco especial de código executado automaticamente quando um objeto é criado com `new`. Sua função é inicializar o objeto em um estado válido. O construtor tem o mesmo nome da classe e **não tem tipo de retorno** — nem mesmo `void`.

Se você não declarar nenhum construtor, o Java implicitamente cria um **construtor padrão** sem parâmetros que não faz nada além de chamar `super()`. Porém, assim que você declara qualquer construtor explicitamente, o Java para de gerar o construtor padrão automaticamente — se ainda precisar de um construtor sem argumentos, você deve declará-lo você mesmo.

A **sobrecarga de construtores** permite que a mesma classe tenha múltiplos construtores com listas de parâmetros diferentes. O compilador escolhe qual chamar com base nos argumentos passados. Isso dá flexibilidade: você pode criar objetos com diferentes níveis de configuração inicial.

```
public class Carro {  
    private String modelo;  
    private int    ano;  
  
    // Construtor padrão (deve ser declarado se houver outro)  
    public Carro() { this("Desconhecido", 2024); } // chaining!  
  
    // Construtor com parâmetros  
    public Carro(String modelo, int ano) {  
        this.modelo = modelo;  
        this.ano    = ano;  
    }  
}
```

this() vs super() — Constructor Chaining

Constructor chaining é a técnica de fazer um construtor chamar outro construtor para evitar duplicação de código de inicialização. Há duas formas:

`this(argumentos)` chama outro construtor **da mesma classe**. `super(argumentos)` chama um construtor **da classe pai**. Ambas as chamadas *devem ser a primeira instrução* do construtor — o compilador rejeita qualquer código antes delas. Um construtor não pode ter tanto `this()` quanto `super()` — são mutuamente exclusivos.

```
public class Retangulo extends Forma {  
    private double largura, altura;  
  
    public Retangulo() {  
        this(0, 0, 1.0, 1.0); // chama o construtor abaixo  
    }  
  
    public Retangulo(int x, int y, double larg, double alt) {  
        super(x, y); // chama construtor de Forma  
        this.largura = larg;  
        this.altura = alt;  
    }  
}
```

■ Constructor Chaining

Constructor chaining reduz código duplicado: o construtor mais completo inicializa todas as variáveis; os demais delegam a ele via `this()`. O pai é sempre inicializado antes pelo `super()`.

3

this vs super

Palavras-chave de acesso a membros da classe atual e da classe pai

Palavras-chave this e super

Além de chamar construtores, `this` e `super` são usadas para acessar **membros** (campos e métodos) da classe atual e da classe pai, respectivamente.

`this.campo` acessa o campo da instância atual — essencial quando um parâmetro tem o mesmo nome do campo (ambiguidade). `super.metodo()` chama a implementação do método na classe pai — útil no `override` quando você quer estender (e não substituir completamente) o comportamento do pai. Nenhuma das duas pode ser usada em contexto estático (`static`), pois não há instância associada.

```
public class Animal {
    protected String nome;
    public void fazerSom() { System.out.println("Som generico"); }
}

public class Cachorro extends Animal {
    public void fazerSom() {
        super.fazerSom();           // executa o som genérico do pai primeiro
        System.out.println("Au!"); // depois adiciona o comportamento próprio
    }
    public void setNome(String nome) {
        this.nome = nome;          // this resolve ambiguidade de nome
    }
}
```

4

Herança

extends, IS-A, reuso de código e override de métodos

Herança em Java

Herança é o mecanismo pelo qual uma classe (**subclasse**) adquire automaticamente todos os campos e métodos não-privados de outra classe (**superclasse**). Em vez de reescrever código, você *reutiliza* o que já foi implementado e apenas adiciona ou modifica o que é específico da subclasse.

A palavra-chave `extends` declara a herança. O relacionamento semântico é de **IS-A**: *Cachorro IS-A Animal* — um Cachorro é um tipo mais específico de Animal. Toda classe em Java que não declara `extends` herda implicitamente de `java.lang.Object` — a raiz de toda hierarquia.

Um ponto importante: campos `private` da superclasse **não são herdados** diretamente — a subclasse não os enxerga. Campos `protected` são herdados e acessíveis nas subclasses. Isso reforça a necessidade de getters e setters para acesso controlado.

```
public class Animal {
    protected String nome;
    public void comer() { System.out.println(nome + " esta comendo"); }
}

public class Cachorro extends Animal {
    private String raca;
    // herda comer() de Animal sem duplicar código
    public void latir() { System.out.println("Au au!"); }
}

// Uso:
Cachorro rex = new Cachorro();
rex.nome = "Rex";    // protected – acessível na subclasse
rex.comer();         // herdado de Animal
rex.latir();         // próprio de Cachorro
```

■ Regra IS-A

IS-A: Cachorro IS-A Animal — a subclasse é um tipo mais específico da superclasse. Sempre verifique se a relação faz sentido semanticamente antes de usar herança. Se não fizer, considere composição.

Override de Método — Sobrescrita

Override (sobrescrita) é criar na subclasse um método com a **mesma assinatura** (nome + parâmetros) de um método da superclasse. Quando você chama esse método em um objeto da subclasse, a versão da subclasse é executada — não a do pai.

A anotação `@Override` não é obrigatória, mas é fortemente recomendada: o compilador verifica se realmente há um método na superclasse com essa assinatura — se você errar o nome ou os parâmetros, o compilador avisa imediatamente em vez de silenciosamente criar um novo método. Métodos `static`, `private` e `final` **não podem** ser sobrescritos.

```
public class Animal {
    public String toString() { return "Animal: " + nome; }
}

public class Cachorro extends Animal {
    @Override
    public String toString() {
        return super.toString() + ", Raca: " + raca;
        // Saída: "Animal: Rex, Raca: Labrador"
    }
}

// Polimorfismo em ação:
Animal a = new Cachorro(); // variável: Animal, objeto real: Cachorro
System.out.println(a);     // chama Cachorro.toString() – não Animal!
```

5

Static vs Instance • POJO • Object

Variáveis estáticas, POJOs e a raiz `java.lang.Object`

Variáveis e Métodos Estáticos vs de Instância

Uma **variável de instância** existe por objeto — cada instância tem sua própria cópia independente. Uma **variável estática** (`static`) pertence à *classe*, não a nenhum objeto específico: existe uma única cópia compartilhada por todas as instâncias.

Da mesma forma, um **método estático** pertence à classe e não tem acesso a `this` nem a campos de instância — só pode acessar membros estáticos. É chamado via `NomeClasse.metodo()`. Um **método de instância** pertence ao objeto e pode acessar tanto campos de instância quanto estáticos. Regra prática: se um método não usa nenhum campo de instância, deve ser estático.

```
public class Contador {
    private static int total = 0; // UMA cópia para toda a classe
    private int id;              // cópia própria de cada objeto

    public Contador() {
        total++;                // incrementa o contador global
        this.id = total;        // id único desta instância
    }

    public static int getTotal() { return total; } // método estático
    public int getId()           { return id;     } // método de instância
}

// Uso:
Contador c1 = new Contador(); // total=1, id=1
Contador c2 = new Contador(); // total=2, id=2
System.out.println(Contador.getTotal()); // 2 — via classe
```

POJO — Plain Old Java Object

Um **POJO** é uma classe simples que serve basicamente como contêiner de dados — sem lógica de negócio complexa, sem dependências de frameworks. Também chamado de **JavaBean**, **Entity** ou **DTO** (Data Transfer Object). É a estrutura mais comum para representar dados que transitam entre camadas da aplicação (ex: dados de um formulário, linha de banco de dados, resposta de API).

Um POJO típico tem: campos private, um construtor (ou vários), getters e setters públicos, e um toString() sobrescrito. IDEs como IntelliJ e Eclipse conseguem gerar automaticamente getters, setters, construtores e toString() a partir dos campos.

java.lang.Object — A Raiz de Tudo

Toda classe em Java herda implicitamente de java.lang.Object — mesmo sem declarar extends. Isso significa que todo objeto tem acesso a métodos como toString(), equals(Object), hashCode() e clone(), herdados de Object. A versão padrão de toString() retorna algo como NomeClasse@1a2b3c — por isso é quase sempre sobrescrita para retornar uma representação legível.

```
public class MinhaClasse { // implicitamente extends Object
    private String valor;

    @Override
    public String toString() {
        return "MinhaClasse{valor='" + valor + "'}";
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof MinhaClasse)) return false;
        return this.valor.equals(((MinhaClasse)obj).valor);
    }
}
```

6

Overloading vs Overriding

Diferenças, regras e covariant return type

Overloading e Overriding são dois conceitos fundamentais que envolvem métodos com o mesmo nome, mas funcionam de formas completamente diferentes. Confundi-los é um erro clássico — a tabela abaixo compara os dois lado a lado.

Nome do método	Mesmo nome	Mesmo nome
Parâmetros	Diferentes	Iguais (mesma assinatura)
Onde ocorre	Mesma classe	Subclasse e superclasse (IS-A)
Tipo de retorno	Pode ser diferente	Igual ou covariante
Modificador de acesso	Qualquer	Não pode ser mais restritivo
Momento de decisão	Compile-time	Runtime (JVM)
Também chamado	Polimorfismo estático	Polimorfismo dinâmico

Regras para Overriding e Covariant Return Type

Para que um método seja reconhecido como override, ele deve ter: o mesmo nome e os mesmos parâmetros (mesma assinatura), e o tipo de retorno deve ser **igual ou covariante** (covariante = subclasse do tipo retornado pelo pai). O modificador de acesso não pode ser mais restritivo que o do pai — se o pai declara `protected`, o filho pode manter `protected` ou aumentar para `public`, mas não reduzir para `private`.

```
// Covariant return type – retorno covariante
class Animal {
    public Animal clone() { return new Animal(); }
}
class Cachorro extends Animal {
    @Override
    public Cachorro clone() { return new Cachorro(); } // Cachorro IS-A Animal OK!
}

// Regra de acesso:
class Pai {
    protected void metodo() { } // protected
}
class Filho extends Pai {
    @Override
    public void metodo() { } // OK: public > protected
    // private void metodo() { } // ERRO: private < protected
}
```

7

Mini-Projects

Exercícios 30–37: Sum, Person, Wall, Point, Carpet, Complex, Cylinder, Cuboid

Os mini-projects a seguir aplicam progressivamente os conceitos desta apostila. Começam simples (classes básicas com métodos) e avançam até herança com super e cálculos geométricos. Tente implementar cada um antes de consultar o esqueleto de código.

Mini-Project 30

Sum Calculator

Implemente uma classe com um método que some dois inteiros. Pratique a criação de uma classe simples com método de instância, instanciação com new e chamada do método. Teste com valores positivos, negativos e zero.

```
public class SumCalculator {
    public int sumDigits(int a, int b) {
        return a + b;
    }
}

// Uso:
SumCalculator calc = new SumCalculator();
System.out.println(calc.sumDigits(3, 5)); // 8
System.out.println(calc.sumDigits(-2, 10)); // 8
```

Mini-Project 31

Person Class + Age Validation

Crie a classe `Person` com campos `name` (`String`) e `age` (`int`), ambos `private`. O setter de idade deve rejeitar valores fora do intervalo 0–100, mantendo o valor anterior se o novo for inválido. Implemente getters para ambos os campos.

```
public class Person {
    private String name;
    private int age;

    public void setAge(int age) {
        if (age >= 0 && age <= 100) this.age = age;
        // fora do range: mantém valor anterior – sem alteração
    }
    public int getAge() { return age; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Mini-Project 32**Wall Area Computation**

Calcule a área de uma parede a partir de altura e largura. Valide no construtor que as dimensões sejam positivas (use 0 se negativas). Implemente o método `calculateArea()` que retorna `height * width`.

```
public class Wall {
    private double height;
    private double width;

    public Wall(double h, double w) {
        this.height = (h > 0) ? h : 0;
        this.width = (w > 0) ? w : 0;
    }
    public double calculateArea() { return height * width; }
}
// Wall wall = new Wall(3.0, 5.5);
// System.out.println(wall.calculateArea()); // 16.5
```

Mini-Project 33**Point Class + Distance**

Implemente a classe `Point` com campos `x` e `y`. O método `distance(Point other)` calcula a distância euclidiana entre dois pontos usando a fórmula: $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$. Use `Math.sqrt()` e `Math.pow()` ou multiplicação direta.

```
public class Point {
    private double x, y;

    public Point(double x, double y) { this.x = x; this.y = y; }

    public double distance(Point other) {
        double dx = this.x - other.x;
        double dy = this.y - other.y;
        return Math.sqrt(dx*dx + dy*dy);
    }
}
// Point p1 = new Point(0,0); Point p2 = new Point(3,4);
// p1.distance(p2); // 5.0
```

Mini-Project 34**Carpet Cost Calculator**

Crie as classes `Floor` (largura e comprimento com `getArea()`) e `Carpet` (custo por m^2). A classe `Calculator` tem o método `total(Floor f, Carpet c)` que retorna `f.getArea() * c.getCost()`. Valide que todos os valores sejam ≥ 0 .

```
public class Floor {
    private double length, width;
    public Floor(double l, double w) {
        length = (l >= 0) ? l : 0; width = (w >= 0) ? w : 0;
    }
    public double getArea() { return length * width; }
}
public class Carpet {
    private double cost;
    public Carpet(double cost) { this.cost = (cost >= 0) ? cost : 0; }
    public double getCost() { return cost; }
}
public class Calculator {
    public double total(Floor f, Carpet c) {
        return f.getArea() * c.getCost();
    }
}
```

Mini-Project 35**Complex Number Operations**

Implemente a classe `Complex` com campos `real` e `imaginary`. Os métodos `add()`, `subtract()` e `multiply()` devem retornar um novo objeto `Complex` com o resultado (imutabilidade). Sobrescreva `toString()` para exibir no formato `'a + bi'`.

```
public class Complex {
    private double real, imaginary;

    public Complex(double real, double imaginary) {
        this.real = real; this.imaginary = imaginary;
    }
    public Complex add(Complex c) {
        return new Complex(real + c.real, imaginary + c.imaginary);
    }
    public Complex subtract(Complex c) {
        return new Complex(real - c.real, imaginary - c.imaginary);
    }
    @Override
    public String toString() { return real + " + " + imaginary + "i"; }
}
```

Mini-Project 36**Cylinder extends Circle**

Crie a classe Circle com o campo radius e o método $area() = \pi \times r^2$. A classe Cylinder herda de Circle e adiciona height. O método $volume() = super.area() \times height$ demonstra o uso de super para reutilizar o cálculo de área sem duplicar código.

```
public class Circle {
    private double radius;
    public Circle(double radius) { this.radius = radius; }
    public double area() { return Math.PI * radius * radius; }
}
public class Cylinder extends Circle {
    private double height;
    public Cylinder(double radius, double height) {
        super(radius); this.height = height;
    }
    public double volume() {
        return super.area() * height; // reutiliza area() do pai!
    }
}
```

Mini-Project 37**Cuboid Pool Area**

Implemente Cuboid herdando de uma classe Rectangle (ou com campos próprios). $getTopSurface() = width \times length$. $getTotalSurface() = 2 \times (w \times h + w \times l + h \times l)$, somando todas as 6 faces do cuboide.

```
public class Cuboid extends Rectangle {
    private double height;

    public Cuboid(double w, double l, double h) {
        super(w, l); this.height = h;
    }
    public double getTopSurface() {
        return getWidth() * getLength();
    }
    public double getTotalSurface() {
        double w = getWidth(), l = getLength();
        return 2 * (w*height + w*l + height*l);
    }
}
```

Resumo — O que você aprendeu nesta apostila

Classe	Blueprint para criar objetos. Define campos (estado) e métodos (comportamento).
Objeto/Instância	Criado com new. Tem sua própria cópia dos campos. Variável armazena referência, não o objeto.
null	Ausência de objeto. Nunca chame métodos em null — NullPointerException.
Encapsulamento	Campos private + getters/setters com validação. Protege o estado interno.
Construtor	Inicializa o objeto. Sem tipo de retorno. Sobrecarga possível. this() e super() encadeiam construtores.
Herança (extends)	Subclasse herda campos e métodos não-private da superclasse. Relação IS-A.
Override (@Override)	Subclasse redefine método do pai com mesma assinatura. Executado em runtime (polimorfismo).
Static vs Instance	static: pertence à classe (uma cópia). Instance: pertence ao objeto (cópia por instância).
POJO	Classe simples de dados. Campos private, getters/setters, toString(). Também chamado JavaBean ou DTO.
Overloading vs Overriding	Overloading: mesmo nome, parâmetros diferentes, mesma classe, compile-time. Overriding: mesma assinatura, subclasse, runtime.